

# recap 19/11/25

|                          |                          |
|--------------------------|--------------------------|
| 🕒 priorité               | moyenne                  |
| 📌 BR01                   | <u>IA41</u>              |
| 📅 date d'accomplissement | @19 novembre 2025        |
| ☑️ fait                  | <input type="checkbox"/> |

some code you experimented with and the results

some insights from what we learned from the lectures that could help our project

1. Algorithm MinMax

2. Alpha-Beta Procedure (Alpha-Beta Pruning)

3. Need for Limited Depth and Evaluation Function

some knowledge you gained about AI algorithm for Gomoku, by reading articles, watching videos etc

some improvement you want to add to our current code

some new ideas for the AI Algorithm for our Gomoku you came up with by thinking, playing Gomoku

some knowledge about the state-of-the-art of Gomoku games that have integrated AI

## **announcement**

I asked the professor whether we could use pygame for the visual implementation and he said yes !

## **some code you experimented with and the results**

I created a function `test_around` that allows to manually input any game configuration, this will help us because we can gain some time when testing specific configs

```
def test_around():  
    board = Board(15)  
    board.coordinates_setter()  
    board.grid[8][7].state = 'black'  
    board.grid[9][8].state = 'black'  
    board.grid[10][9].state = 'black'  
    board.grid[11][10].state = 'black'
```

```
board.grid[3][10].state = 'white'  
board.grid[4][9].state = 'white'  
board.grid[5][8].state = 'white'  
board.grid[6][7].state = 'white'
```

```
return board
```

I created a function `start_directly` that allows us to start directly with some configuration "hard-coded", so we don't have to answer all the questions at the start of the program.

This will allow us to go straight to testing and not waste some time

```
def start_directly_2(self):  
    self.players.append(Player("Alice"))  
    self.players[0].color = 'white'  
    self.board = tests.test_around()  
    self.board.third_display()  
  
    gomoku_AI : AI = AI(2)  
    gomoku_AI.set_tables(self.board)  
  
    gomoku_AI.update_white_around(self.board, 8, 7)  
    gomoku_AI.update_white_around(self.board, 9, 8)  
    gomoku_AI.update_white_around(self.board, 10, 9)  
    gomoku_AI.update_white_around(self.board, 11, 10)  
  
    gomoku_AI.update_black_around(self.board, 3, 10)  
    gomoku_AI.update_black_around(self.board, 4, 9)  
    gomoku_AI.update_black_around(self.board, 5, 8)  
    gomoku_AI.update_black_around(self.board, 6, 7)  
  
    gomoku_AI.display_tables(self.board)
```

|    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10      | 11      | 12      | 13      | 14      | 15      |
|----|--------|--------|--------|--------|--------|--------|--------|--------|--------|---------|---------|---------|---------|---------|---------|
| 1  | (1,1)  | (1,2)  | (1,3)  | (1,4)  | (1,5)  | (1,6)  | (1,7)  | (1,8)  | (1,9)  | (1,10)  | (1,11)  | (1,12)  | (1,13)  | (1,14)  | (1,15)  |
| 2  | (2,1)  | (2,2)  | (2,3)  | (2,4)  | (2,5)  | (2,6)  | (2,7)  | (2,8)  | (2,9)  | (2,10)  | (2,11)  | (2,12)  | (2,13)  | (2,14)  | (2,15)  |
| 3  | (3,1)  | (3,2)  | (3,3)  | (3,4)  | (3,5)  | (3,6)  | (3,7)  | (3,8)  | (3,9)  | (3,10)  | (3,11)  | (3,12)  | (3,13)  | (3,14)  | (3,15)  |
| 4  | (4,1)  | (4,2)  | (4,3)  | (4,4)  | (4,5)  | (4,6)  | (4,7)  | (4,8)  | (4,9)  | (4,10)  | ●       | (4,12)  | (4,13)  | (4,14)  | (4,15)  |
| 5  | (5,1)  | (5,2)  | (5,3)  | (5,4)  | (5,5)  | (5,6)  | (5,7)  | (5,8)  | (5,9)  | ●       | (5,11)  | (5,12)  | (5,13)  | (5,14)  | (5,15)  |
| 6  | (6,1)  | (6,2)  | (6,3)  | (6,4)  | (6,5)  | (6,6)  | (6,7)  | (6,8)  | ●      | (6,10)  | (6,11)  | (6,12)  | (6,13)  | (6,14)  | (6,15)  |
| 7  | (7,1)  | (7,2)  | (7,3)  | (7,4)  | (7,5)  | (7,6)  | (7,7)  | ●      | (7,9)  | (7,10)  | (7,11)  | (7,12)  | (7,13)  | (7,14)  | (7,15)  |
| 8  | (8,1)  | (8,2)  | (8,3)  | (8,4)  | (8,5)  | (8,6)  | (8,7)  | (8,8)  | (8,9)  | (8,10)  | (8,11)  | (8,12)  | (8,13)  | (8,14)  | (8,15)  |
| 9  | (9,1)  | (9,2)  | (9,3)  | (9,4)  | (9,5)  | (9,6)  | (9,7)  | ●      | (9,9)  | (9,10)  | (9,11)  | (9,12)  | (9,13)  | (9,14)  | (9,15)  |
| 10 | (10,1) | (10,2) | (10,3) | (10,4) | (10,5) | (10,6) | (10,7) | (10,8) | ●      | (10,10) | (10,11) | (10,12) | (10,13) | (10,14) | (10,15) |
| 11 | (11,1) | (11,2) | (11,3) | (11,4) | (11,5) | (11,6) | (11,7) | (11,8) | (11,9) | ●       | (11,11) | (11,12) | (11,13) | (11,14) | (11,15) |
| 12 | (12,1) | (12,2) | (12,3) | (12,4) | (12,5) | (12,6) | (12,7) | (12,8) | (12,9) | (12,10) | ●       | (12,12) | (12,13) | (12,14) | (12,15) |
| 13 | (13,1) | (13,2) | (13,3) | (13,4) | (13,5) | (13,6) | (13,7) | (13,8) | (13,9) | (13,10) | (13,11) | (13,12) | (13,13) | (13,14) | (13,15) |
| 14 | (14,1) | (14,2) | (14,3) | (14,4) | (14,5) | (14,6) | (14,7) | (14,8) | (14,9) | (14,10) | (14,11) | (14,12) | (14,13) | (14,14) | (14,15) |
| 15 | (15,1) | (15,2) | (15,3) | (15,4) | (15,5) | (15,6) | (15,7) | (15,8) | (15,9) | (15,10) | (15,11) | (15,12) | (15,13) | (15,14) | (15,15) |

```

def update_white_around (self, board : Board, row : int, col : int) :
    dict0 = {1 : 110, 2 : 25, 3 : 5, 4 : 1}
    dict = {1 : 1, 2 : 1, 3 : 1, 4 : 1}
    self.white_around[row][col] = '○'
    directions = [(1,0), (0,1), (1,1), (-1,1)]
    for vertical, horizontal in directions:
        for i in range(1,5):
            new_x = row + i * vertical
            new_y = col + i * horizontal
            if (0 <= new_x < board.size) and (0 <= new_y < board.size):
                if board.grid[new_x][new_y].state == None:
                    self.white_around[new_x][new_y] += dict[i]
            new_x = row - i * vertical

```

```

new_y = col - i * horizontal
if (0 <= new_x < board.size) and (0 <= new_y < board.size):
    if board.grid[new_x][new_y].state == None:
        self.white_around[new_x][new_y] += dict[i]

```

```

def update_black_around (self, board : Board, row : int, col : int) :
    dict0 = {1 : 110, 2 : 25, 3 : 5, 4 : 1}
    dict = {1 : 1, 2 : 1, 3 : 1, 4 : 1}
    self.black_around[row][col] = '●'
    directions = [(1,0), (0,1), (1,1), (-1,1)]
    for vertical, horizontal in directions:
        for i in range(1,5):
            new_x = row + i * vertical
            new_y = col + i * horizontal
            if (0 <= new_x < board.size) and (0 <= new_y < board.size):
                if board.grid[new_x][new_y].state == None:
                    self.black_around[new_x][new_y] += dict[i]
            new_x = row - i * vertical
            new_y = col - i * horizontal
            if (0 <= new_x < board.size) and (0 <= new_y < board.size):
                if board.grid[new_x][new_y].state == None:
                    self.black_around[new_x][new_y] += dict[i]

```

**some insights from what we learned from the lectures that could help our project**

An  **$m,n,k$ -game** is a game in which two players take turns in placing a stone of their color on an  **$m$ -by- $n$**  board, the winner being the player who first gets  **$k$**  stones of their own color in a row, horizontally, vertically, or diagonally

## GOMOKU

- *two-player*
- *zero-sum : Games in which the "algebraic" sum of players' winnings is constant. What one player wins is necessarily lost by another.*
- *full and perfect information (SNICP) games : Complete information, each player has comprehensive knowledge when making decisions. The player knows:*
  - *Their own possible actions.*
  - *The possible actions of other players.*
  - *The gains resulting from these actions.*
  - *The motivations of other players.*

*Perfect information : has a sequential mechanism where each player has detailed knowledge of all the actions taken before making their own choice. These games are also referred to as asynchronous or alternating, as players make their decisions one after the other, with information about their opponents' moves.*

## How to solve the game ?

Determine a sequence of winning movements to reach a final position, using algorithms based on the exploration of a **game tree**.

The main algorithms for games in situations of adversity are the MinMax algorithm and the Alpha-Beta procedure.

### 1. Algorithm MinMax

The MinMax algorithm is based on the principle of **maximizing the utility value** for your player (MAX) by assuming that your opponent (MIN) will always play to minimize this value.

It works in three stages:

1. **Extend the game tree.**
2. **Evaluate each terminal node** (final position).
3. **Propagate these values** to non-terminal nodes, taking the minimum value for MIN player nodes and the maximum value for MAX player

nodes.

However, the *time complexity* of MinMax is  $O(b^m)$ , where **b** is the branching factor (number of possible moves per state) and **m** is the depth of the tree. **Given the potentially gigantic size of the game tree for interesting games**, the MinMax algorithm is often **intractable**.

## 2. Alpha-Beta Procedure (Alpha-Beta Pruning)

The most common algorithm used to solve SNICP games is the Alpha-Beta pruning. This is an optimization of MinMax that aims to **reduce the search space** by:

- Extending the game tree to depth  $h$  by a deep search.
- **No longer exploring the successors of a node as soon as it is certain that this node will not be chosen, given the nodes already examined.**

Pruning retains the quality of the result while being much more efficient. In the best case, Alpha-Beta evaluates a fraction of the nodes that MinMax evaluates, which allows for double the search depth

## 3. Need for Limited Depth and Evaluation Function

For an AI playing a  $m,n,k$  game that could be complex (big  $m$ ,  $n$ , or  $k$ ), the exploration of the game tree must be done with limited depth (algorithm horizon) in order to provide a decision within a reasonable time.

In this case, you will need an evaluation function (eval) that fulfills several roles:

- It assigns a value to each position (node) at the limit depth.
- It gives an estimate of the outcome of the game for MAX from this position.
- It must integrate specific strategic elements to the game  $m,n,k$  (for example, the number of lines of  $k-1$  tokens blocked, center control, etc.) in order to prioritize favorable positions.

The choice of blow will then be made by applying MinMax (or Alpha-Beta) with a limited depth ( $N$ ) and backpropagating the values of the leaf nodes (obtained via the evaluation function) to the root.

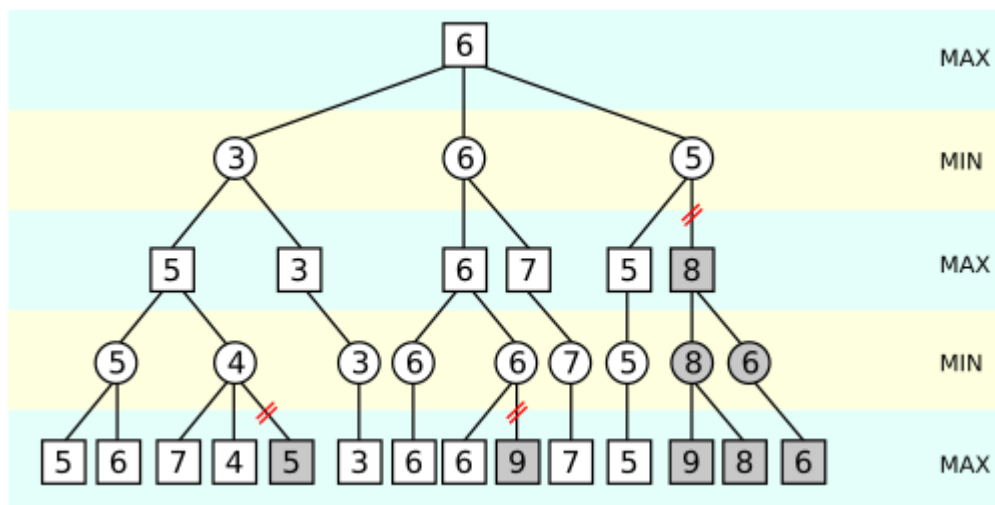
## Formalization of the Problem

Before applying MinMax or Alpha-Beta, the problem must be formalized by

defining:

1. The state: How to code a game position (for example, an  $m \times n$  matrix representing the board).
2. The initial state and the final states (victory/defeat situations).
3. The production system (or transition function): The set of rules that allow to legally generate the set of states that can be reached from the current state.

In summary, to create your game AI  $m,n,k$ , the algorithm of choice is the Alpha-Beta Procedure coupled with limited depth search and a relevant evaluation function for intermediate positions.



## Les Valeurs Alpha ( $\alpha$ ) et Beta ( $\beta$ )

Chaque nœud exploré dans l'arbre de recherche conserve la trace de deux valeurs:  $\alpha$  et  $\beta$ .

1. **Valeur Alpha ( $\alpha$ )** : Représente la valeur du **meilleur successeur** trouvé jusqu'à présent pour un nœud MAX. C'est une limite inférieure sur la valeur qui sera remontée.

2. **Valeur Beta ( $\beta$ )** : Représente la valeur du **pire successeur** trouvé jusqu'à présent pour un nœud MIN. C'est une limite supérieure sur la valeur qui sera remontée.

Les valeurs initiales du nœud racine sont  $\alpha = -\infty$  et  $\beta = +\infty$ . Les valeurs  $\alpha$  et  $\beta$  du parent d'un nœud  $N$  sont mises à jour lorsque toute la recherche en dessous de  $N$  est terminée ou a été interrompue.

## Les Règles d'Élagage

L'élagage permet de conserver la qualité du résultat tout en étant plus efficace que MinMax. Les règles d'interruption de la recherche (l'élagage lui-même) sont les suivantes :

- **Interruption pour un nœud MAX :** La recherche est interrompue sous un nœud MAX  $N$  si sa valeur  $\alpha$  est supérieure ou égale à la  $\beta$  valeur de son nœud parent ( $\alpha(N) \geq \beta(\text{parent})$ ).
- **Interruption pour un nœud MIN :** La recherche est interrompue sous un nœud MIN  $N$  si sa valeur  $\beta$  est inférieure ou égale à la  $\alpha$  valeur d'un ancêtre de  $N$  de type MAX ( $\beta(N) \leq \alpha(\text{ancêtre MAX})$ ).

### Formulation Récursive Détaillée

Les sources décrivent la structure récursive de l'algorithme Alpha-Beta :  
**Fonction MaxValue(e, p,  $\alpha$ ,  $\beta$ )** (où  $e$  est l'état,  $p$  la profondeur,  $\alpha$  et  $\beta$  les limites courantes) :

1. Si  $e$  est terminal OU  $p=0$ , Retourner `eval(e)` .
2. Initialiser  $v \leftarrow -\infty$ .
3. Pour chaque successeur  $s$  de  $e$  Faire :
  - $v \leftarrow \max(v, \text{MinValue}(s, p-1, \alpha, \beta))$
  - **Si  $v \geq \beta$  Alors Retourner  $v$**  (Ceci est l'élagage).
  - $\alpha \leftarrow \max(\alpha, v)$
4. Retourner  $v$ .

**Fonction MinValue(e, p,  $\alpha$ ,  $\beta$ ) :**

1. Si  $e$  est terminal OU  $p=0$ , Retourner `eval(e)` .
2. Initialiser  $v \leftarrow +\infty$ .
3. Pour chaque successeur  $s$  de  $e$  Faire :
  - $v \leftarrow \min(v, \text{MaxValue}(s, p-1, \alpha, \beta))$
  - **Si  $v \leq \alpha$  Alors Retourner  $v$**  (Ceci est l'élagage).
  - $\beta \leftarrow \min(\beta, v)$
4. Retourner  $v$ .

L'efficacité de cet élagage dépend de l'ordre d'apparition des nœuds successeurs. Idéalement, il faut examiner d'abord les coups qui ont le plus de chance de générer des nœuds à forte valeur d'évaluation pour un nœud MAX. Dans le meilleur des cas, Alpha-Beta permet de **doubler la profondeur de recherche** par rapport à MinMax.

Un exemple graphique est mentionné dans les sources sous le titre "Exemple d'élagage Alpha-Beta" pour illustrer le comportement de l'algorithme sur un graphe de recherche.

**some knowledge you gained about AI algorithm for Gomoku, by reading articles, watching videos etc**

<https://en.wikipedia.org/wiki/Gomoku>

Researchers have been applying artificial intelligence techniques to playing gomoku for several decades. Joseph Weizenbaum published a short paper in Datamation in 1962 entitled "How to Make a Computer

Appear Intelligent"[27], that described the strategy used in a gomoku program that could beat novice players. In 1994, L. Victor Allis raised the algorithm of proof-number search (pn-search) and dependency-based search (db-search), and proved that when starting from an empty  $15 \times 15$  board, the first player has a winning strategy using these searching algorithms.[28] This applies to both free-style gomoku and standard gomoku without any opening rules. It seems very likely that black wins on larger boards too. In any size of a board, freestyle gomoku is an  $m, n, k$ -game, hence it is known that the first player can force a win or a draw. In 2001, Allis's winning strategy was also approved for renju, a variation of gomoku, when there was no limitation on the opening stage. [29]

However, neither the theoretical values of all legal positions, nor the opening rules such as Swap2 used by the professional gomoku players have been solved yet, so the topic of gomoku artificial intelligence is still a challenge for computer scientists, such as the problem on how to improve the gomoku algorithms to make them more strategic and competitive. Most state-of-the-art gomoku algorithms are based on the alpha-beta pruning framework.[citation needed]

Reisch proved that Generalized gomoku is PSPACE-complete. [30] He also observed that the reduction can be adapted to the rules of  $k$ -in-a-Row for fixed  $k$ . Although he did not specify exactly which values of  $k$  are allowed, the reduction would appear to generalize to any  $k \geq 5$ . [31]

There exist several well-known tournaments for gomoku programs since 1989. The Computer Olympiad started with the gomoku game in 1989, but gomoku has not been in the list since 1993.[32] The Renju World Computer Championship was started in 1991, and held for 4 times until 2004.[33][34] The Gomocup tournament has been held annually since 2000, with more than thirty participants from about ten countries taking part.[35] The Hungarian Computer Go-Moku Tournament was played twice in 2005.[36][37] There were also two Computer vs. Human tournaments played in the Czech Republic, in 2006 and 2011.[38][39] Not until 2017 did the computer programs prove capable of outperforming the world human champion in public competitions. In the Gomoku World Championship 2017, there was a match between the world champion program Yixin and the world champion human player Rudolf Dupszki. Yixin won the match with a score of 2-0.[40][41]

<https://ebiquity.umbc.edu/paper/html/id/1130/How-to-Make-a-Computer-Appear-Intelligent>

<http://fragrieu.free.fr/SearchingForSolutions.pdf>

### **some improvement you want to add to our current code**

- separate the different functions into independant modules
- for each function, create some documentation using this format :

```

"""
    Updates the black stones adjacency table based on a new stone placement.
    Uses directional vectors to check in all four directions.
    Limits the search to 4 squares away from the placed stone.
"""

def update_black_around (self, board : Board, row : int, col : int) :
    dict0 = {1 : 110, 2 : 25, 3 : 5, 4 : 1}
    dict = {1 : 1, 2 : 1, 3 : 1, 4 : 1}
    self.black_around[row][col] = '●'
    directions = [(1,0), (0,1), (1,1), (-1,1)]
    for vertical, horizontal in directions:
        for i in range(1,5):
            new_x = row + i * vertical
            new_y = col + i * horizontal
            if (0 <= new_x < board.size) and (0 <= new_y < board.size):
                if board.grid[new_x][new_y].state == None:
                    self.black_around[new_x][new_y] += dict[i]
            new_x = row - i * vertical
            new_y = col - i * horizontal
            if (0 <= new_x < board.size) and (0 <= new_y < board.size):
                if board.grid[new_x][new_y].state == None:
                    self.black_around[new_x][new_y] += dict[i]

```

maybe add what all the parameters correspond to if necessary

**some new ideas for the AI Algorithm for our Gomoku you came up with by thinking, playing Gomoku**

**some knowledge about the state-of-the-art of Gomoku games that have integrated AI**

- they use Alpha-Beta pruning algorithm
- they use Min-Max algorithm

<https://github.com/lihongxun945/gobang>

